

Top1-Top2 Fark Analizi ile Cevap Doğruluğu Tahmini

Hesaplamalı Anlambilim Dersi – Final Projesi

Model: Cosmos-T1 (`ytu-ce-cosmos/Turkish-Gemma-9b-T1`)

Dataset: gsm8k_tr (`ytu-ce-cosmos/gsm8k_tr`)

Yöntem: Modelin her token için top1-top2 olasılık farkını alıp, bundan istatistiksel özellikler çıkararak cevabın doğru/yanlış olduğunu ML ile tahmin etmek

GPU: A100

Akış

```
gsm8k_tr → Cosmos-T1 (vLLM, logprobs=2) → top1-top2 fark dizisi +  
→ Dengeli dataset (1000 train + 100 test) → Özellik çıkarm  
→ Confusion Matrix + Metrikler + Feature Importance
```

0. Oturum Kurulumu (Her Restart Sonrası Çalıştır)

```

from google.colab import drive
import os, gc

drive.mount('/content/drive')

ROOT = '/content/drive/MyDrive/top1top2_proje'
os.makedirs(ROOT, exist_ok=True)

PATHS = {
    'root'      : ROOT,
    'data'      : f'{ROOT}/data',
    'generations' : f'{ROOT}/generations',
    'results'   : f'{ROOT}/results',
    'figures'   : f'{ROOT}/figures',
}
for p in PATHS.values():
    os.makedirs(p, exist_ok=True)

import json, re, numpy as np
print('Drive bağlandı. ROOT:', ROOT)

```

Mounted at /content/drive
Drive bağlandı. ROOT: /content/drive/MyDrive/top1top2_proje

✓ 1. Kurulum

```

# Transformers modu: vLLM kullanılmıyor.
!pip install -q -U transformers accelerate datasets scikit-learn sc
print("Kurulum tamam. Paket güncellendiyse Runtime > Restart runtime

```

```

_____ 62.1/62.1 kB 7.3 MB/s
_____ 80.3/80.3 kB 9.5 MB/s
_____ 11.2/11.2 MB 121.4 MB/s
_____ 389.2/389.2 kB 41.0 MB/s
_____ 555.1/555.1 kB 51.9 MB/s
_____ 9.1/9.1 MB 144.1 MB/s et
_____ 35.2/35.2 MB 71.4 MB/s e
_____ 10.0/10.0 MB 133.9 MB/s
_____ 48.9/48.9 MB 51.2 MB/s e
Kurulum tamam. Paket güncellendiyse Runtime > Restart runtime yapıp

```

```
import torch
print('GPU:', torch.cuda.get_device_name(0))
print('VRAM:', f"{torch.cuda.get_device_properties(0).total_memory
```

```
GPU: NVIDIA A100-SXM4-40GB
VRAM: 42 GB
```

✓ 2. Yardımcı Fonksiyonlar

```

def _normalize_num(tok):
    tok = tok.strip()
    has_dot, has_com = '.' in tok, ',' in tok
    if has_dot and has_com:
        if tok.rfind(',') > tok.rfind('.'):
            tok = tok.replace('.', '').replace(',', '.')
        else:
            tok = tok.replace(',', '.')
    elif has_dot:
        p = tok.split('.')
        if len(p) > 2 or (len(p) == 2 and len(p[1]) == 3 and len(p[1]) > 1):
            tok = tok.replace('.', '')
    elif has_com:
        p = tok.split(',')
        if len(p) == 2 and len(p[1]) == 3:
            tok = tok.replace(',', '.')
        else:
            tok = tok.replace(',', '.')
    try:
        v = float(tok)
        return int(v) if v == int(v) else v
    except:
        return None

def extract_number(text):
    """#### formatından sayı çıkar; yoksa son sayıyı dene."""
    m = re.findall(r'####\s*([\-+]?[\\d.\\+])', text)
    if m:
        v = _normalize_num(m[-1])
        if v is not None:
            return v
    nums = re.findall(r'[\-+]?\\d[\\d.]*', text)
    for n in reversed(nums):
        v = _normalize_num(n)
        if v is not None:
            return v
    return None

def is_correct(pred, gold, tol=1e-3):
    if pred is None or gold is None:
        return False
    return abs(float(pred) - float(gold)) <= tol

print('Yardımcı fonksiyonlar hazır.')

```

Yardımcı fonksiyonlar hazır.

3. Dataset Hazırlama

```
from datasets import load_dataset
import random, json

random.seed(42)
np.random.seed(42)

ds = load_dataset('ytu-ce-cosmos/gsm8k_tr')
print(ds)

# train split kullan
ds_train = ds['train']
print(f'Train split: {len(ds_train)} örnek')
print('Sütunlar:', ds_train.column_names)
print('\nİlk örnek:')
print('SORU:', ds_train[0]['question'])
print('CEVAP:', ds_train[0]['answer'][:200])
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your se
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Ple
WARNING:huggingface_hub.utils._http:Warning: You are sending unauth
DatasetDict({
  train: Dataset({
    features: ['question', 'answer'],
    num_rows: 8792
  })
})
```

Train split: 8792 örnek
Sütunlar: ['question', 'answer']

İlk örnek:

SORU: Borris tekel bayisi her 6 ayda bir 90 kilogram üzüm kullanıyor
CEVAP: Borris şu anda her 6 ayda 90 kilogram üzüm kullanıyorsa, bir

$180 \text{ kilogram} \times 1.20 = 216 \text{ kilogram}$

Buna göre, üretimi

```
# Gold cevabı olan tüm örnekleri al ve karıştır
temiz = []
for i, ex in enumerate(ds_train):
    g = extract_number(ex['answer'])
    if g is None:
        continue
    temiz.append({
        'id'      : i,
        'question': ex['question'].strip(),
        'answer'  : ex['answer'].strip(),
        'gold'    : g,
    })

print(f'Temiz örnek: {len(temiz)} (atılan: {len(ds_train) - len(temiz)})')
random.shuffle(temiz)

# Tüm havuzu kaydet – sonra dengeli subset seçeceğiz
with open(f"{PATHS['data']}/all_questions.json", 'w', encoding='utf-8') as f:
    json.dump(temiz, f, ensure_ascii=False, indent=2)
print('Kaydedildi:', f"{PATHS['data']}/all_questions.json")
```

```
Temiz örnek: 8768 (atılan: 24)
Kaydedildi: /content/drive/MyDrive/top1top2_proje/data/all_questions.json
```

✓ 4. Cosmos-T1 ile Üretim (vLLM + logprobs=2)

Anahtar: `SamplingParams(logprobs=2)` ile her token adımımda top-2 token ve log-olasılıklarını alıyoruz.

Top1-Top2 **olasılık farkı** = $\exp(\text{logprob_top1}) - \exp(\text{logprob_top2})$

Bu fark büyükse model emin, küçükse kararsız demektir.

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_NAME = 'ytu-ce-cosmos/Turkish-Gemma-9b-T1'

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, trust_remote_
tokenizer.padding_side = 'left'
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype=torch.bfloat16,
    device_map='auto',
    trust_remote_code=True,
)
model.eval()

print('Model yüklendi:', MODEL_NAME)
print('VRAM kullanımı:', f"{torch.cuda.memory_allocated()/1e9:.1f}")
```

```
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
[transformers] The following generation flags are not valid and may
Loading weights: 100% 464/464 [00:04<00:00, 103.50it/s]

Model yüklendi: ytu-ce-cosmos/Turkish-Gemma-9b-T1
VRAM kullanımı: 18.5 GB
```

```

SYSTEM = (
    'Sen matematik problemleri çözen bir asistansın. '
    'Problemi adım adım Türkçe çöz. '
    'Cevabın en sonunda mutlaka şu formatta nihai sayıyı yaz: ####
)

def build_prompt(question):
    messages = [
        {'role': 'system', 'content': SYSTEM},
        {'role': 'user', 'content': question},
    ]
    return tokenizer.apply_chat_template(
        messages, tokenize=False, add_generation_prompt=True
    )

# Test
print(build_prompt('5 + 3 kaçtır?')[:200])

```

```

<bos><start_of_turn>system
Sen matematik problemleri çözen bir asistansın. Problemi adım adım T
<start_of_tu

```

```
print('Prompt builder hazır (transformers modu).')
```

```
Prompt builder hazır (transformers modu).
```

```

import math, time

MAX_NEW_TOKENS = 512
BATCH_SIZE      = 8

def top_diffs_from_scores(scores):
    diffs = []
    for step_logits in scores:
        probs = torch.softmax(step_logits[0].float(), dim=-1) # .f
        top2 = torch.topk(probs, k=2)
        diffs.append((top2.values[0] - top2.values[1]).item())
    return diffs

def generate_batch(questions_batch):
    prompts = [build_prompt(q['question']) for q in questions_batch]
    enc = tokenizer(
        prompts,
        return_tensors='pt',
        padding=True,

```

```

        truncation=True,
        max_length=1024,
    ).to(model.device)

    with torch.no_grad():
        out = model.generate(
            **enc,
            max_new_tokens=MAX_NEW_TOKENS,
            do_sample=False,
            return_dict_in_generate=True,
            output_scores=True,
            pad_token_id=tokenizer.pad_token_id,
        )

    prompt_len = enc['input_ids'].shape[1]
    results = []
    for i in range(len(questions_batch)):
        gen_ids = out.sequences[i, prompt_len:]
        gen_text = tokenizer.decode(gen_ids, skip_special_tokens=True)
        n_gen = (gen_ids != tokenizer.pad_token_id).sum().item()
        step_scores = [out.scores[t][i] for t in range(n_gen)]
        diffs = top_diffs_from_scores(step_scores)
        results.append((gen_text, diffs))
    return results

def run_batch(questions_subset, batch_size=BATCH_SIZE, save_path=None):
    if save_path and os.path.exists(save_path):
        with open(save_path, encoding='utf-8') as f:
            done = json.load(f)
            done_ids = {r['id'] for r in done}
            print(f'Checkpoint yüklendi: {len(done)} sonuç')
    else:
        done, done_ids = [], set()

    remaining = [q for q in questions_subset if q['id'] not in done_ids]
    print(f'Kalan: {len(remaining)} soru')

    for start in range(0, len(remaining), batch_size):
        batch = remaining[start:start + batch_size]
        t0 = time.time()
        try:
            batch_results = generate_batch(batch)
        except Exception as e:
            print(f'Hata (batch {start}): {e}')
            continue

        for q, (gen_text, diffs) in zip(batch, batch_results):

```

```
pred = extract_number(gen_text)
done.append({
    'id'          : q['id'],
    'question'    : q['question'],
    'answer'      : q['answer'],
    'gold'        : q['gold'],
    'pred'        : pred,
    'is_correct'  : bool(is_correct(pred, q['gold'])),
    'top_diffs'   : diffs,
    'n_tokens'    : len(diffs),
    'gen_text'    : gen_text,
})

if save_path:
    tmp_path = save_path + '.tmp'
    with open(tmp_path, 'w', encoding='utf-8') as f:
        json.dump(done, f, ensure_ascii=False)
    os.replace(tmp_path, save_path)

n_done = start + len(batch)
n_correct_batch = sum(r['is_correct'] for r in done[-len(batch):])
print(f' [{n_done}/{len(remaining)}] {time.time()-t0:.1f} '
      f'Doğru: {n_correct_batch}/{len(batch)}')
```

return done

print('Fonksiyonlar hazır.')

Fonksiyonlar hazır.

```
# Küçük test – 5 soru ile logprob akışını doğrula
test5 = temiz[:5]
batch_out = generate_batch(test5)

for q, (gen_text, diffs) in zip(test5, batch_out):
    pred = extract_number(gen_text)
    print(f"GOLD={q['gold']} PRED={pred} DOĞRU={is_correct(pred,
        f"N_TOKEN={len(diffs)} MEAN_DIFF={np.mean(diffs):.3f} M
    print(f" Cevap: {gen_text[:100]}")
    print()
```

```
GOLD=39 PRED=39 DOĞRU=True N_TOKEN=512 MEAN_DIFF=0.885 MIN=0.00
Cevap: <think>
Öncelikle, soruyu anlamaya çalışıyorum. "Violet'in Tickletoe'nun iki

GOLD=600 PRED=10 DOĞRU=False N_TOKEN=512 MEAN_DIFF=0.820 MIN=0.
Cevap: <think>
Öncelikle, problemde bir çelişki var gibi görünüyor. İlk cümle: "Bir

GOLD=67500 PRED=45000 DOĞRU=False N_TOKEN=512 MEAN_DIFF=0.906 M
Cevap: <think>
John sanat koleksiyonu yapıyor. İlk üç sanat eseri için aynı fiyatı

GOLD=21 PRED=21 DOĞRU=True N_TOKEN=512 MEAN_DIFF=0.809 MIN=0.00
Cevap: <think>
Elizabeth'in 10 yeniden kullanılabilir su şişesi var. Önce okulda 2

GOLD=4000 PRED=3 DOĞRU=False N_TOKEN=512 MEAN_DIFF=0.876 MIN=0.
Cevap: <think>
Öncelikle, problemde bir kavanozda jelibonlar olduğu ve bunların dör
```

✓ 5. Tüm Sorular Üzerinde Çalıştır

A100 ile ~3000 soru **15-20 dakikada** biter (batch_size=64).

```
GEN_PATH = f"{PATHS['generations']}/cosmos_t1_all.json"

FAST_MODE = True # Önce pilot. Final için False yap.
N_RUN = min(300, len(temiz)) if FAST_MODE else min(3000, len(temiz))
RUN_BATCH_SIZE = 4 if FAST_MODE else BATCH_SIZE

print('FAST_MODE:', FAST_MODE)
print('N_RUN:', N_RUN, 'batch:', RUN_BATCH_SIZE)
all_results = run_batch(
    questions_subset=temiz[:N_RUN],
```

```

        batch_size=RUN_BATCH_SIZE,
        save_path=GEN_PATH,
    )

n_correct = sum(r['is_correct'] for r in all_results)
print(f'\nToplam: {len(all_results)}')
print(f'Doğru: {n_correct} ({100*n_correct/max(1,len(all_results))})')
print(f'Yanlış: {len(all_results)-n_correct}')

```

```

FAST_MODE: True
N_RUN: 300 batch: 4
Checkpoint yüklendi: 80 sonuç
Kalan: 220 soru
[4/220] 105.6s Doğru: 0/4
[8/220] 95.6s Doğru: 1/4
[12/220] 93.1s Doğru: 0/4
[16/220] 90.4s Doğru: 1/4
[20/220] 95.6s Doğru: 4/4
[24/220] 89.6s Doğru: 2/4
[28/220] 95.6s Doğru: 0/4
[32/220] 92.4s Doğru: 0/4
[36/220] 116.3s Doğru: 1/4
[40/220] 98.2s Doğru: 2/4
[44/220] 95.7s Doğru: 2/4
[48/220] 90.2s Doğru: 0/4
[52/220] 93.2s Doğru: 1/4
[56/220] 101.8s Doğru: 1/4
[60/220] 88.8s Doğru: 1/4
[64/220] 101.9s Doğru: 1/4
[68/220] 97.9s Doğru: 1/4
[72/220] 92.2s Doğru: 3/4
[76/220] 101.1s Doğru: 0/4
[80/220] 97.3s Doğru: 2/4
[84/220] 94.8s Doğru: 2/4
[88/220] 89.2s Doğru: 2/4
[92/220] 97.2s Doğru: 0/4
[96/220] 103.4s Doğru: 1/4
[100/220] 101.1s Doğru: 1/4
[104/220] 88.9s Doğru: 1/4
[108/220] 98.7s Doğru: 0/4
[112/220] 92.1s Doğru: 3/4
[116/220] 98.5s Doğru: 4/4
[120/220] 91.8s Doğru: 0/4
[124/220] 93.3s Doğru: 1/4
[128/220] 95.0s Doğru: 1/4
[132/220] 108.9s Doğru: 0/4
[136/220] 94.3s Doğru: 1/4
[140/220] 95.2s Doğru: 0/4
[144/220] 96.1s Doğru: 2/4
[148/220] 86.6s Doğru: 2/4
[152/220] 97.3s Doğru: 1/4
[156/220] 111.1s Doğru: 0/4

```

```
[160/220] 93.7s Doğru: 0/4
[164/220] 105.9s Doğru: 0/4
[168/220] 97.0s Doğru: 1/4
[172/220] 94.8s Doğru: 0/4
[176/220] 90.6s Doğru: 1/4
[180/220] 95.7s Doğru: 2/4
[184/220] 102.3s Doğru: 1/4
[188/220] 101.2s Doğru: 1/4
[192/220] 99.7s Doğru: 2/4
[196/220] 93.8s Doğru: 0/4
[200/220] 96.0s Doğru: 1/4
[204/220] 94.2s Doğru: 1/4
[208/220] 93.9s Doğru: 1/4
[212/220] 97.6s Doğru: 1/4
[216/220] 88.4s Doğru: 1/4
[220/220] 82.0s Doğru: 2/4
```

```
# temiz listesini Drive'a kaydet (bir kez çalıştır)
with open(f'{PATHS['data']}/temiz.json', 'w', encoding='utf-8') as f:
    json.dump(temiz, f, ensure_ascii=False)
print(f'Kaydedildi: {len(temiz)} soru')
```

Kodlamaya başlayın veya yapay zeka ile kod [oluşturun](#).

✓ 6. Dengeli Dataset Oluşturma

Ödev şartı: eğitim ≥ 1000 , test ≥ 100 , sınıf dağılımları eşit.

```
import json, numpy as np, pandas as pd
GEN_PATH_FINAL = f'{PATHS['generations']}/cosmos_t1_final_l4.json'
with open(GEN_PATH_FINAL, encoding="utf-8") as f:
    all_results = json.load(f)
print(f"Yüklendi: {len(all_results)} sonuç")
```

Yüklendi: 1200 sonuç

```
# Yeni checkpoint'ten yükle
import json
GEN_PATH_FINAL = f"{PATHS['generations']}/cosmos_t1_final_l4.json"
with open(GEN_PATH_FINAL, encoding="utf-8") as f:
    all_results = json.load(f)

n_correct = sum(r["is_correct"] for r in all_results)
n_wrong = len(all_results) - n_correct
print(f"Toplam: {len(all_results)}")
print(f"Doğru: {n_correct} ({100*n_correct/len(all_results):.1f}%)
print(f"Yanlış: {n_wrong}")
```

```
Toplam: 1200
Doğru: 370 (30.8%)
Yanlış: 830
```

```

import pandas as pd

df = pd.DataFrame(all_results)
df = df[df['top_diffs'].apply(lambda x: len(x) >= 5)].copy()

correct_df = df[df['is_correct'] == True].sample(frac=1, random_state=42)
wrong_df = df[df['is_correct'] == False].sample(frac=1, random_state=42)

print(f'Kullanılabilir doğru: {len(correct_df)}')
print(f'Kullanılabilir yanlış: {len(wrong_df)}')

TRAIN_PER_CLASS = 320 # 370 doğru var, 50 testi ayır → 320 train
TEST_PER_CLASS = 50
PER_CLASS = TRAIN_PER_CLASS + TEST_PER_CLASS # = 370

assert len(correct_df) >= PER_CLASS, f'Yeterli doğru yok! ({len(correct_df)} < {PER_CLASS})'
assert len(wrong_df) >= PER_CLASS, f'Yeterli yanlış yok! ({len(wrong_df)} < {PER_CLASS})'

train_df = pd.concat([
    correct_df.iloc[:TRAIN_PER_CLASS],
    wrong_df.iloc[:TRAIN_PER_CLASS],
]).sample(frac=1, random_state=42).reset_index(drop=True)

test_df = pd.concat([
    correct_df.iloc[TRAIN_PER_CLASS:PER_CLASS],
    wrong_df.iloc[TRAIN_PER_CLASS:PER_CLASS],
]).sample(frac=1, random_state=42).reset_index(drop=True)

print(f'\nEğitim: {len(train_df)} (doğru={train_df["is_correct"].sum()})')
print(f'Test: {len(test_df)} (doğru={test_df["is_correct"].sum()})')

train_df.to_json(f'{PATHS["data"]}/train_balanced_final.json', orient='records')
test_df.to_json(f'{PATHS["data"]}/test_balanced_final.json', orient='records')
print('Kaydedildi.')

```

```

Kullanılabilir doğru: 370
Kullanılabilir yanlış: 830

```

```

Eğitim: 640 (doğru=320, yanlış=320)
Test: 100 (doğru=50, yanlış=50)
Kaydedildi.

```

```

import pandas as pd

df = pd.DataFrame(all_results)
df = df[df['top_diffs'].apply(lambda x: len(x) >= 5)].copy()

```

```

correct_df = df[df['is_correct'] == True].sample(frac=1, random_state=42)
wrong_df = df[df['is_correct'] == False].sample(frac=1, random_state=42)

print(f'Kullanılabilir doğru: {len(correct_df)}')
print(f'Kullanılabilir yanlış: {len(wrong_df)}')

FAST_MODE = globals().get('FAST_MODE', True)

if FAST_MODE:
    TRAIN_PER_CLASS = 50
    TEST_PER_CLASS = 10
else:
    TRAIN_PER_CLASS = 500
    TEST_PER_CLASS = 50

PER_CLASS = TRAIN_PER_CLASS + TEST_PER_CLASS

assert len(correct_df) >= PER_CLASS, f'Yeterli doğru yok! ({len(correct_df)} < {PER_CLASS})'
assert len(wrong_df) >= PER_CLASS, f'Yeterli yanlış yok! ({len(wrong_df)} < {PER_CLASS})'

train_df = pd.concat([
    correct_df.iloc[:TRAIN_PER_CLASS],
    wrong_df.iloc[:TRAIN_PER_CLASS],
]).sample(frac=1, random_state=42).reset_index(drop=True)

test_df = pd.concat([
    correct_df.iloc[TRAIN_PER_CLASS:PER_CLASS],
    wrong_df.iloc[TRAIN_PER_CLASS:PER_CLASS],
]).sample(frac=1, random_state=42).reset_index(drop=True)

print(f'\nEğitim seti: {len(train_df)} (doğru={train_df["is_correct"].sum()})')
print(f'Test seti: {len(test_df)} (doğru={test_df["is_correct"].sum()})')

train_df.to_json(f'{PATHS["data"]}/train_balanced.json', orient='records')
test_df.to_json(f'{PATHS["data"]}/test_balanced.json', orient='records')
print('Dengeli dataset kaydedildi.')

```

```

Kullanılabilir doğru: 92
Kullanılabilir yanlış: 208

```

```

Eğitim seti: 100 (doğru=50, yanlış=50)
Test seti: 20 (doğru=10, yanlış=10)
Dengeli dataset kaydedildi.

```

✓ 7. Özellik Çıkarma

Her soru için top1-top2 fark dizisinden istatistiksel özellikler.

```

from scipy import stats as sp_stats

def extract_features(diffs):
    arr = np.array(diffs, dtype=float)
    n = len(arr)

    feat = {
        'mean'          : np.mean(arr),
        'median'        : np.median(arr),
        'std'           : np.std(arr),
        'min'           : np.min(arr),
        'max'           : np.max(arr),
        'q25'           : np.percentile(arr, 25),
        'q75'           : np.percentile(arr, 75),
        'iqr'           : np.percentile(arr, 75) - np.percentile(arr, 25),
        'skewness'       : float(sp_stats.skew(arr)),
        'kurtosis'       : float(sp_stats.kurtosis(arr)),
        'prop_high'      : np.mean(arr > 0.5),
        'prop_low'       : np.mean(arr < 0.1),
        'prop_medium'    : np.mean((arr >= 0.1) & (arr <= 0.5)),
        'mean_first10'   : np.mean(arr[:10]) if n >= 10 else np.mean(arr),
        'mean_last10'    : np.mean(arr[-10:]) if n >= 10 else np.mean(arr),
        'trend'          : (np.mean(arr[-10:]) - np.mean(arr[:10])) if n >= 20 else 0,
        'n_tokens'       : float(n),
        'min_region_mean': np.mean(arr[max(0, np.argmin(arr)-3):np.argmax(arr)+3])
    }

    return {k: float(np.nan_to_num(v, nan=0.0, posinf=0.0, neginf=0.0)) for k, v in feat.items()}

def build_XY(df_subset):
    feats = [extract_features(row['top_diffs']) for _, row in df_subset.iterrows()]
    X = pd.DataFrame(feats)
    y = df_subset['is_correct'].astype(int).values
    return X, y

X_train, y_train = build_XY(train_df)
X_test, y_test = build_XY(test_df)

FEATURE_NAMES = X_train.columns.tolist()
print('Özellik sayısı:', len(FEATURE_NAMES))
print('X_train:', X_train.shape, ' X_test:', X_test.shape)

Özellik sayısı: 18
X_train: (640, 18) X_test: (100, 18)

```

✓ 8. ML Modelleri Eğitimi ve Karşılaştırması

```

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientB
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, ConfusionMatrixDisplay, roc_auc
)

MODELS = {
    'Logistic Regression': Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('sc', StandardScaler()),
        ('clf', LogisticRegression(max_iter=2000, C=1.0, random
    ])),
    'Random Forest': Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('clf', RandomForestClassifier(
            n_estimators=300, max_depth=12, min_samples_leaf=3,
            random_state=42, n_jobs=-1,
        )),
    ]),
    'Gradient Boosting': Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('clf', GradientBoostingClassifier(
            n_estimators=300, learning_rate=0.05, max_depth=5,
            subsample=0.8, random_state=42,
        )),
    ]),
}

ml_results = {}

for name, clf in MODELS.items():
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    y_prob = clf.predict_proba(X_test)[:, 1]

    ml_results[name] = {
        'Accuracy' : accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred, zero_division=
        'Recall' : recall_score(y_test, y_pred, zero_division=0),
        'F1' : f1_score(y_test, y_pred, zero_division=0),
        'ROC-AUC' : roc_auc_score(y_test, y_prob),
        'y_pred' : y_pred,
        'y_prob' : y_prob,
        'clf' : clf,

```

```

    }
    print(f'{name}: Acc={ml_results[name]["Accuracy"]:.4f}   '
          f'F1={ml_results[name]["F1"]:.4f}   AUC={ml_results[name] [

```

```

Logistic Regression: Acc=0.6700   F1=0.7027   AUC=0.7560
Random Forest: Acc=0.7400   F1=0.7679   AUC=0.8032
Gradient Boosting: Acc=0.7300   F1=0.7523   AUC=0.7916

```

9. Karşılaştırma Tablosu

```

import pandas as pd

tablo = pd.DataFrame({
    name: {
        'Accuracy' : f"{v['Accuracy']:.4f}",
        'Precision': f"{v['Precision']:.4f}",
        'Recall'    : f"{v['Recall']:.4f}",
        'F1-score'  : f"{v['F1']:.4f}",
        'ROC-AUC'   : f"{v['ROC-AUC']:.4f}",
    }
    for name, v in ml_results.items()
}).T

print('='*65)
print('          MODEL KARŞILAŞTIRMA TABLOSU')
print('='*65)
print(tablo.to_string())
print('='*65)

best = max(ml_results, key=lambda k: ml_results[k]['F1'])
print(f'\nEn iyi model (F1): {best} → F1={ml_results[best]["F1"]}:

```

```

=====
          MODEL KARŞILAŞTIRMA TABLOSU
=====

```

	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.6700	0.6393	0.7800	0.7027	0.7560
Random Forest	0.7400	0.6935	0.8600	0.7679	0.8032
Gradient Boosting	0.7300	0.6949	0.8200	0.7523	0.7916

```

=====

```

```

En iyi model (F1): Random Forest → F1=0.7679

```

10. Confusion Matrix

```

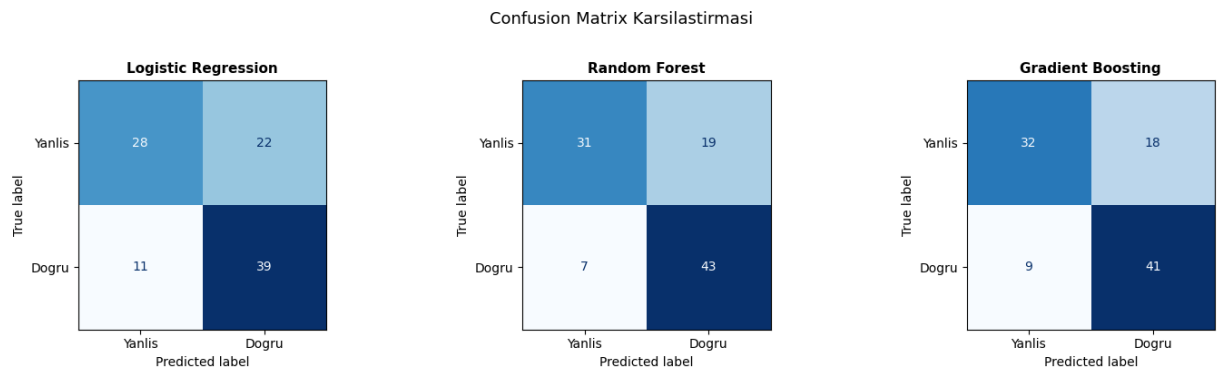
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'DejaVu Sans'

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, (name, v) in zip(axes, ml_results.items()):
    cm = confusion_matrix(y_test, v['y_pred'])
    ConfusionMatrixDisplay(cm, display_labels=['Yanlis', 'Dogru']).
        ax=ax, colorbar=False, cmap='Blues'
    )
    ax.set_title(name, fontsize=11, fontweight='bold')

plt.suptitle('Confusion Matrix Karsilastirmasi', fontsize=13, y=1.0)
plt.tight_layout()
plt.savefig(f"{PATHS['figures']}/confusion_matrices.png", dpi=150,
plt.show()

```



✓ 11. ROC Eğrisi

```

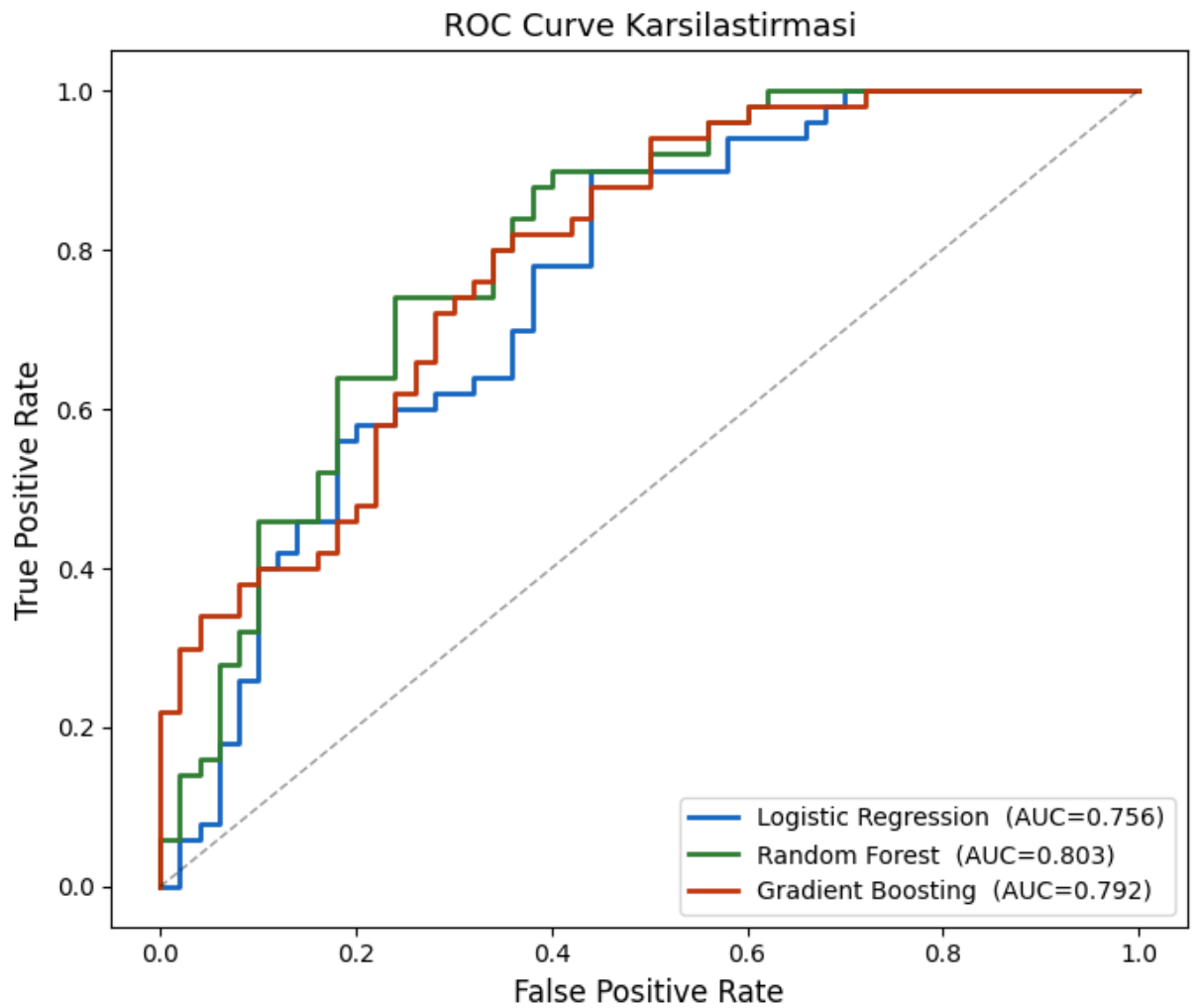
plt.figure(figsize=(7, 6))
colors = ['#1565C0', '#2E7D32', '#BF360C']

for (name, v), color in zip(ml_results.items(), colors):
    fpr, tpr, _ = roc_curve(y_test, v['y_prob'])
    plt.plot(fpr, tpr, lw=2, color=color,
             label=f"{name} (AUC={v['ROC-AUC']:.3f})")

plt.plot([0,1],[0,1], 'k--', alpha=0.4, lw=1)
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)

```

```
plt.title('ROC Curve Karsilastirmasi', fontsize=13)
plt.legend(fontsize=10)
plt.tight_layout()
plt.savefig(f"{PATHS['figures']}/roc_curves.png", dpi=150)
plt.show()
```



✓ 12. Feature Importance

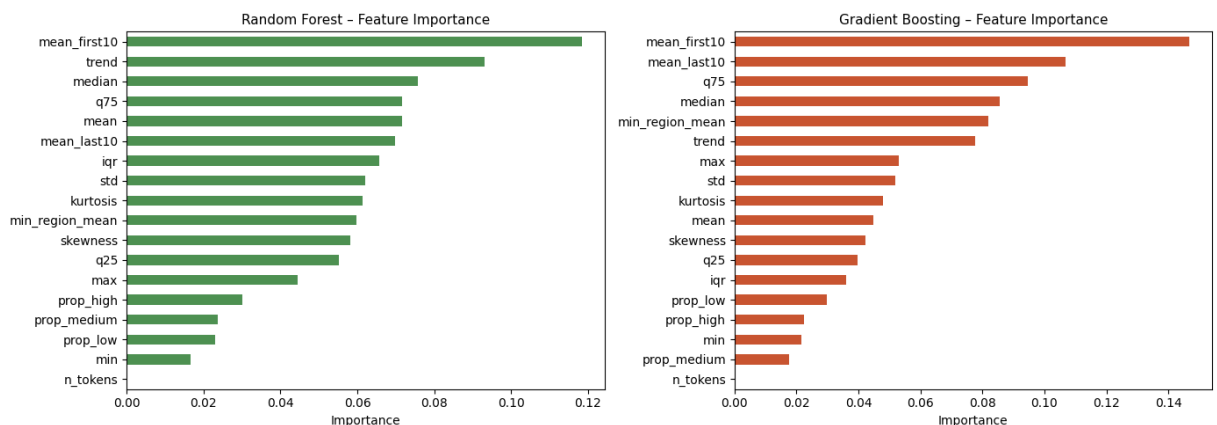
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, model_name, color in zip(
    axes,
    ['Random Forest', 'Gradient Boosting'],
    ['#2E7D32', '#BF360C']):

    clf = ml_results[model_name]['clf'].named_steps['clf'] # pipel
    imp = pd.Series(clf.feature_importances_, index=FEATURE_NAMES).
    imp.plot(kind='barh', ax=ax, color=color, alpha=0.85)
    ax.set_title(f'{model_name} - Feature Importance', fontsize=11)
    ax.set_xlabel('Importance')

plt.tight_layout()
plt.savefig(f"{PATHS['figures']}/feature_importance.png", dpi=150)
plt.show()

print('\n=== En Önemli 5 Özellik (Gradient Boosting) ===')
gb_clf = ml_results['Gradient Boosting']['clf'].named_steps['clf']
gb_imp = pd.Series(gb_clf.feature_importances_, index=FEATURE_NAMES)
print(gb_imp.head(5).to_string())
```



```
=== En Önemli 5 Özellik (Gradient Boosting) ===
mean_first10      0.146739
mean_last10      0.106784
q75              0.094698
median           0.085590
min_region_mean  0.081803
```

✓ 13. Top1–Top2 Fark Dağılımının Görselleştirilmesi

```

import seaborn as sns

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# 1) Ortalama fark histogramı
c_means = [np.mean(r) for r in test_df[test_df['is_correct']==True]]
w_means = [np.mean(r) for r in test_df[test_df['is_correct']==False]]
axes[0].hist(c_means, bins=20, alpha=0.65, color='#2E7D32', label='Dogru')
axes[0].hist(w_means, bins=20, alpha=0.65, color='#C62828', label='Yanlis')
axes[0].set_xlabel('Ortalama Top1-Top2 Farki', fontsize=11)
axes[0].set_ylabel('Soru Sayisi')
axes[0].set_title('Ortalama Fark Dagilimi')
axes[0].legend()

# 2) Violin plot – sınıfa göre ortalama fark
plot_df = pd.DataFrame({
    'mean_diff' : c_means + w_means,
    'label'      : ['Dogru']*len(c_means) + ['Yanlis']*len(w_means)
})
sns.violinplot(data=plot_df, x='label', y='mean_diff',
               palette={'Dogru':'#2E7D32','Yanlis':'#C62828'}, ax=axes[1])
axes[1].set_title('Violin Plot')
axes[1].set_xlabel('')

# 3) Örnek token bazlı fark grafiği
sample_c = test_df[test_df['is_correct']==True].iloc[0]['top_diffs']
sample_w = test_df[test_df['is_correct']==False].iloc[0]['top_diffs']
axes[2].plot(sample_c, alpha=0.8, color='#2E7D32', label='Dogru cevap')
axes[2].plot(sample_w, alpha=0.8, color='#C62828', label='Yanlis cevap')
axes[2].set_xlabel('Token Numarasi')
axes[2].set_ylabel('Top1 - Top2 Farki')
axes[2].set_title('Token Bazli Fark (Ornek)')
axes[2].legend()

plt.suptitle('Top1-Top2 Fark Analizi', fontsize=13, y=1.02)
plt.tight_layout()
plt.savefig(f"{PATHS['figures']}/top1_top2_analysis.png", dpi=150,
plt.show()

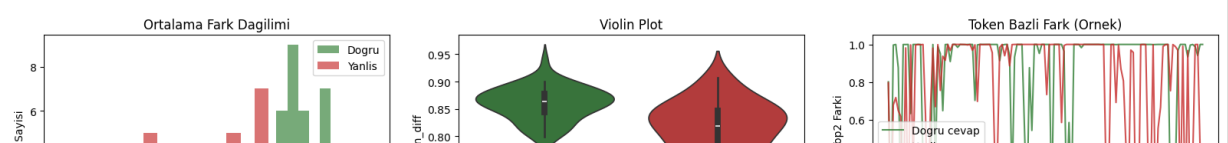
```

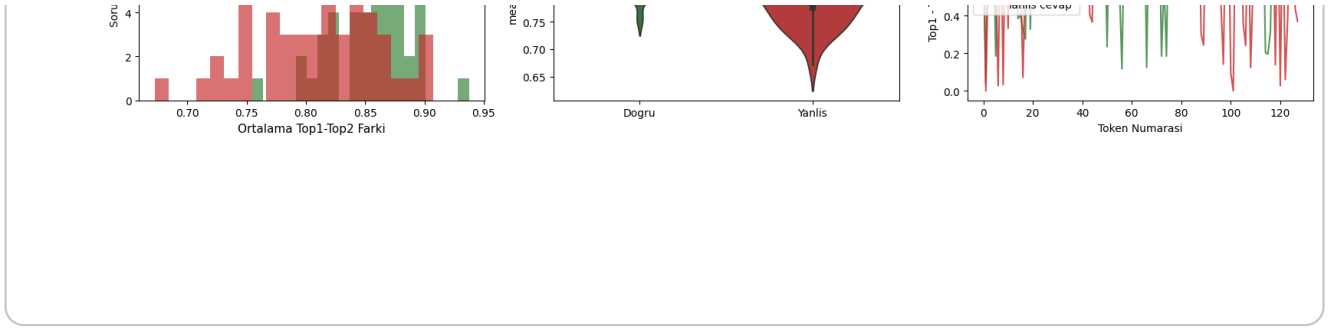
/tmp/ipykernel_430/1567658480.py:20: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be

```
sns.violinplot(data=plot_df, x='label', y='mean_diff',
```

Top1-Top2 Fark Analizi





✓ 14. Sonuçların Drive'a Kaydedilmesi

```
# Tablo ve metrikler JSON olarak kaydet
MODEL_NAME = 'ytu-ce-cosmos/Turkish-Gemma-9b-T1'

summary = {
    'dataset'      : 'ytu-ce-cosmos/gsm8k_tr',
    'model'        : MODEL_NAME ,
    'n_train'      : len(train_df),
    'n_test'       : len(test_df),
    'features'     : FEATURE_NAMES,
    'results'      : {
        name: {
            k: round(v, 6) for k, v in vals.items()
            if k not in ('y_pred', 'y_prob', 'clf')
        }
        for name, vals in ml_results.items()
    },
    'best_model'   : best,
}

summary_path = f"{PATHS['results']}/summary.json"
with open(summary_path, 'w', encoding='utf-8') as f:
    json.dump(summary, f, ensure_ascii=False, indent=2)

print('Kaydedildi:', summary_path)
print()
print('=== ÖZET ===')
print(tablo.to_string())
print(f'\nEn iyi model: {best}')
```

Kaydedildi: /content/drive/MyDrive/top1top2_proje/results/summary.js

=== ÖZET ===

	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.6700	0.6393	0.7800	0.7027	0.7560
Random Forest	0.7400	0.6935	0.8600	0.7679	0.8032
Gradient Boosting	0.7300	0.6949	0.8200	0.7523	0.7916

En iyi model: Random Forest

```

import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_NAME = "ytu-ce-cosmos/Turkish-Gemma-9b-T1"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, trust_remote_
tokenizer.padding_side = "left"

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype=torch.bfloat16,
    device_map="auto",
    trust_remote_code=True,
)

model.eval()

print("Model yüklendi:", MODEL_NAME)
print("GPU:", torch.cuda.get_device_name(0))
print("VRAM:", f"{torch.cuda.memory_allocated()/1e9:.1f} GB")

```

```

config.json: 100% 853/853 [00:00<00:00, 104kB/s]
tokenizer_config.json: 100% 46.8k/46.8k [00:00<00:00, 5.63MB/s]
tokenizer.json: 100% 34.4M/34.4M [00:02<00:00, 15.3MB/s]
special_tokens_map.json: 100% 644/644 [00:00<00:00, 89.4kB/s]
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
model.safetensors.index.json: 100% 39.1k/39.1k [00:00<00:00, 4.78MB/s]

Download complete: 100% 18.5G/18.5G [00:59<00:00, 269MB/s]

Fetching 4 files: 100% 4/4 [00:59<00:00, 15.47s/it]
[transformers] The following generation flags are not valid and may
Loading weights: 100% 464/464 [00:04<00:00, 107.29it/s]

generation_config.json: 100% 223/223 [00:00<00:00, 30.8kB/s]

Model yüklendi: ytu-ce-cosmos/Turkish-Gemma-9b-T1
GPU: NVIDIA A100-SXM4-40GB
VRAM: 18.5 GB

```

```

# =====
# 12001DEN DEVAM. ETNAL TCTN EEA DOCDU : EEA YANITC HERFET

```

```

# 1200 DEN DEVAM: FINAL İÇİN 550 DOĞRU + 550 YANLIŞ HEDEFİ
# =====

import os, json, time, gc
import numpy as np
import torch
from transformers import LogitsProcessor

FAST_MODE = False

MAX_NEW_TOKENS = 128
RUN_BATCH_SIZE = 8

TARGET_CORRECT = 550
TARGET_WRONG = 550

# 1200 yetmedi; checkpointten devam edip 1900'e kadar dener.
N_RUN = min(1900, len(temiz))

GEN_PATH = f"{PATHS['generations']}/cosmos_t1_final_l4.json"

def build_prompt(question):
    return f"""Aşağıdaki matematik probleminin yalnızca nihai sayıya
    Uzun açıklama yazma.
    Son satırı kesinlikle şu formatta yaz:
    #### sayı

    Soru:
    {question}

    Cevap: """"

class Top2Recorder(LogitsProcessor):
    def __init__(self, batch_size):
        self.records = [[] for _ in range(batch_size)]

    def __call__(self, input_ids, scores):
        probs = torch.softmax(scores.float(), dim=-1)
        top2 = torch.topk(probs, k=2, dim=-1)
        diffs = (top2.values[:, 0] - top2.values[:, 1]).tolist()
        for i, d in enumerate(diffs):
            self.records[i].append(d)
        return scores

def generate_batch_fast(questions_batch):
    prompts = [build_prompt(q["question"]) for q in questions_batch]

    enc = tokenizer(
        prompts.

```

```

        prompt,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=1024,
    ).to(model.device)

    recorder = Top2Recorder(batch_size=len(questions_batch))

    with torch.no_grad():
        out = model.generate(
            **enc,
            max_new_tokens=MAX_NEW_TOKENS,
            do_sample=False,
            logits_processor=[recorder],
            pad_token_id=tokenizer.pad_token_id,
        )

    prompt_len = enc["input_ids"].shape[1]
    batch_results = []

    for i in range(len(questions_batch)):
        gen_ids = out[i, prompt_len:]
        gen_text = tokenizer.decode(gen_ids, skip_special_tokens=True)
        n_gen = (gen_ids != tokenizer.pad_token_id).sum().item()
        diffs = recorder.records[i][:n_gen]
        batch_results.append((gen_text, diffs))

    return batch_results

if os.path.exists(GEN_PATH):
    with open(GEN_PATH, encoding="utf-8") as f:
        done = json.load(f)
else:
    done = []

done_ids = {r["id"] for r in done}

questions_subset = temiz[:N_RUN]
remaining = [q for q in questions_subset if q["id"] not in done_ids]

print("Checkpoint:", GEN_PATH)
print("Mevcut sonuç:", len(done))
print("Kalan soru:", len(remaining))
print("N_RUN:", N_RUN)

t0 = time.time()

for start in range(0, len(remaining), RUN_BATCH_SIZE):

```

```

batch = remaining[start:start + RUN_BATCH_SIZE]

batch_out = generate_batch_fast(batch)

for q, (gen_text, diffs) in zip(batch, batch_out):
    pred = extract_number(gen_text)
    correct = is_correct(pred, q["gold"])

    done.append({
        "id": q["id"],
        "question": q["question"],
        "answer": q["answer"],
        "gold": q["gold"],
        "pred": pred,
        "is_correct": bool(correct),
        "gen_text": gen_text,
        "top_diffs": [float(x) for x in diffs],
        "n_tokens": len(diffs),
        "mean_diff": float(np.mean(diffs)) if len(diffs) else None,
        "min_diff": float(np.min(diffs)) if len(diffs) else None
    })

tmp_path = GEN_PATH + ".tmp"
with open(tmp_path, "w", encoding="utf-8") as f:
    json.dump(done, f, ensure_ascii=False)
os.replace(tmp_path, GEN_PATH)

n_correct = sum(r["is_correct"] for r in done)
n_wrong = len(done) - n_correct
elapsed = (time.time() - t0) / 60

print(
    f"[{len(done)}/{N_RUN}] "
    f"Doğru={n_correct} Yanlış={n_wrong} "
    f"Süre={elapsed:.1f} dk"
)

if n_correct >= TARGET_CORRECT and n_wrong >= TARGET_WRONG:
    print("Final için yeterli dengeli veri toplandı.")
    break

torch.cuda.empty_cache()
gc.collect()

all_results = done

n_correct = sum(r["is_correct"] for r in all_results)
n_wrong = len(all_results) - n_correct

```

```

print("\n=== SON DURUM ===")
print("Toplam:", len(all_results))
print("Doğru:", n_correct)
print("Yanlış:", n_wrong)

if n_correct >= 550 and n_wrong >= 550:
    print("Tamam: 1000 train + 100 test için yeterli veri var.")
else:
    print("Henüz yetmedi. Doğru sayısı:", n_correct, "/ 550")

```

Checkpoint: /content/drive/MyDrive/top1top2_proje/generations/cosm

Mevcut sonuç: 1200

Kalan soru: 1632

N_RUN: 1900

```

[1208/1900] Doğru=374 Yanlış=834 Süre=0.5 dk
[1216/1900] Doğru=375 Yanlış=841 Süre=0.9 dk
[1224/1900] Doğru=379 Yanlış=845 Süre=1.3 dk
[1232/1900] Doğru=381 Yanlış=851 Süre=1.8 dk
[1240/1900] Doğru=385 Yanlış=855 Süre=2.2 dk
[1248/1900] Doğru=388 Yanlış=860 Süre=2.7 dk
[1256/1900] Doğru=389 Yanlış=867 Süre=3.1 dk
[1264/1900] Doğru=392 Yanlış=872 Süre=3.4 dk
[1272/1900] Doğru=394 Yanlış=878 Süre=3.9 dk
[1280/1900] Doğru=400 Yanlış=880 Süre=4.3 dk
[1288/1900] Doğru=402 Yanlış=886 Süre=4.7 dk
[1296/1900] Doğru=405 Yanlış=891 Süre=5.1 dk
[1304/1900] Doğru=407 Yanlış=897 Süre=5.5 dk
[1312/1900] Doğru=408 Yanlış=904 Süre=6.0 dk
[1320/1900] Doğru=412 Yanlış=908 Süre=6.4 dk
[1328/1900] Doğru=417 Yanlış=911 Süre=6.8 dk
[1336/1900] Doğru=420 Yanlış=916 Süre=7.4 dk
[1344/1900] Doğru=423 Yanlış=921 Süre=7.8 dk
[1352/1900] Doğru=426 Yanlış=926 Süre=8.4 dk
[1360/1900] Doğru=426 Yanlış=934 Süre=8.8 dk
[1368/1900] Doğru=427 Yanlış=941 Süre=9.4 dk
[1376/1900] Doğru=429 Yanlış=947 Süre=9.8 dk
[1384/1900] Doğru=430 Yanlış=954 Süre=10.4 dk
[1392/1900] Doğru=434 Yanlış=958 Süre=10.8 dk
[1400/1900] Doğru=435 Yanlış=965 Süre=11.3 dk
[1408/1900] Doğru=437 Yanlış=971 Süre=11.7 dk
[1416/1900] Doğru=439 Yanlış=977 Süre=12.2 dk
[1424/1900] Doğru=440 Yanlış=984 Süre=12.6 dk
[1432/1900] Doğru=443 Yanlış=989 Süre=13.1 dk
[1440/1900] Doğru=443 Yanlış=997 Süre=13.7 dk
[1448/1900] Doğru=447 Yanlış=1001 Süre=14.1 dk
[1456/1900] Doğru=449 Yanlış=1007 Süre=14.5 dk
[1464/1900] Doğru=452 Yanlış=1012 Süre=14.9 dk
[1472/1900] Doğru=453 Yanlış=1019 Süre=15.4 dk
[1480/1900] Doğru=458 Yanlış=1022 Süre=15.7 dk
[1488/1900] Doğru=460 Yanlış=1028 Süre=16.2 dk
[1496/1900] Doğru=464 Yanlış=1032 Süre=16.5 dk

```

```
[1504/1900] Doğru=466 Yanlış=1038 Süre=17.0 dk
[1512/1900] Doğru=468 Yanlış=1044 Süre=17.5 dk
[1520/1900] Doğru=470 Yanlış=1050 Süre=18.0 dk
[1528/1900] Doğru=472 Yanlış=1056 Süre=18.4 dk
[1536/1900] Doğru=475 Yanlış=1061 Süre=18.8 dk
[1544/1900] Doğru=477 Yanlış=1067 Süre=19.3 dk
[1552/1900] Doğru=480 Yanlış=1072 Süre=19.8 dk
[1560/1900] Doğru=482 Yanlış=1078 Süre=20.2 dk
[1568/1900] Doğru=486 Yanlış=1082 Süre=20.7 dk
[1576/1900] Doğru=487 Yanlış=1089 Süre=21.1 dk
[1584/1900] Doğru=487 Yanlış=1097 Süre=21.6 dk
[1592/1900] Doğru=493 Yanlış=1099 Süre=22.0 dk
[1600/1900] Doğru=494 Yanlış=1106 Süre=22.4 dk
[1608/1900] Doğru=497 Yanlış=1111 Süre=22.9 dk
[1616/1900] Doğru=498 Yanlış=1118 Süre=23.3 dk
[1624/1900] Doğru=500 Yanlış=1124 Süre=23.7 dk
[1632/1900] Doğru=500 Yanlış=1132 Süre=24.2 dk
```

```
import pandas as pd

df = pd.DataFrame(all_results)
df = df[df["top_diffs"].apply(lambda x: len(x) >= 5)].copy()

correct_df = df[df["is_correct"] == True].sample(frac=1, random_state=42)
wrong_df = df[df["is_correct"] == False].sample(frac=1, random_state=42)

print(f"Kullanılabilir doğru: {len(correct_df)}")
print(f"Kullanılabilir yanlış: {len(wrong_df)}")

TRAIN_PER_CLASS = 500
TEST_PER_CLASS = 50
PER_CLASS = TRAIN_PER_CLASS + TEST_PER_CLASS

assert len(correct_df) >= PER_CLASS, f"Yeterli doğru yok! ({len(correct_df)} < {PER_CLASS})"
assert len(wrong_df) >= PER_CLASS, f"Yeterli yanlış yok! ({len(wrong_df)} < {PER_CLASS})"

train_df = pd.concat([
    correct_df.iloc[:TRAIN_PER_CLASS],
    wrong_df.iloc[:TRAIN_PER_CLASS],
]).sample(frac=1, random_state=42).reset_index(drop=True)

test_df = pd.concat([
    correct_df.iloc[TRAIN_PER_CLASS:PER_CLASS],
    wrong_df.iloc[TRAIN_PER_CLASS:PER_CLASS],
]).sample(frac=1, random_state=42).reset_index(drop=True)

print(f"\nEğitim seti: {len(train_df)} (doğru={train_df['is_correct'].sum()})")
print(f"Test seti: {len(test_df)} (doğru={test_df['is_correct'].sum()})")

train_df.to_json(f"{PATHS['data']}/train_balanced_final.json", orient='records')
test_df.to_json(f"{PATHS['data']}/test_balanced_final.json", orient='records')

print("Final dengeli dataset kaydedildi.")
```

Kullanılabilir doğru: 551
Kullanılabilir yanlış: 1225

Eğitim seti: 1000 (doğru=500, yanlış=500)
Test seti: 100 (doğru=50, yanlış=50)
Final dengeli dataset kaydedildi.

```

import matplotlib.pyplot as plt
import matplotlib
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

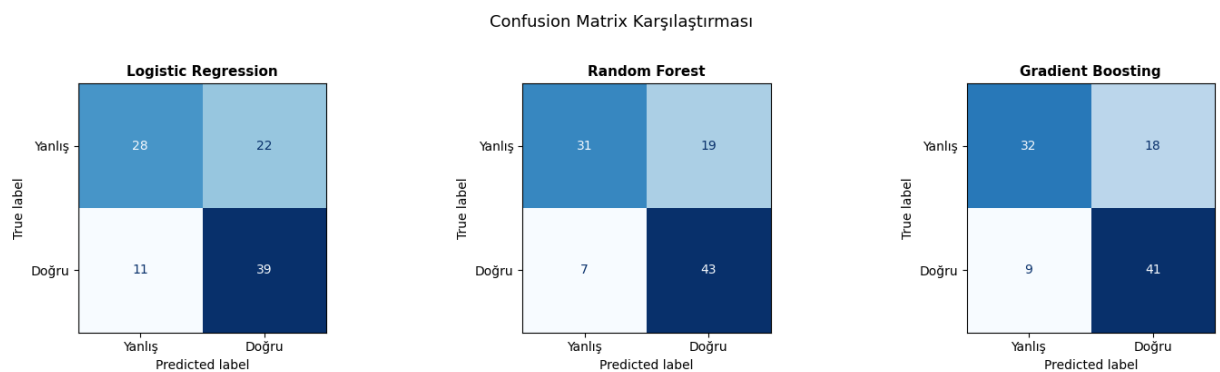
matplotlib.rcParams['font.family'] = 'DejaVu Sans'

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, (name, v) in zip(axes, ml_results.items()):
    cm = confusion_matrix(y_test, v['y_pred'])
    ConfusionMatrixDisplay(cm, display_labels=['Yanlış', 'Doğru']).p
        ax=ax, colorbar=False, cmap='Blues'
    )
    ax.set_title(name, fontsize=11, fontweight='bold')

plt.suptitle('Confusion Matrix Karşılaştırması', fontsize=13, y=1.02)
plt.tight_layout()
plt.savefig(f"{PATHS['figures']}/confusion_matrices_final.png", dpi=
plt.show()

```



```

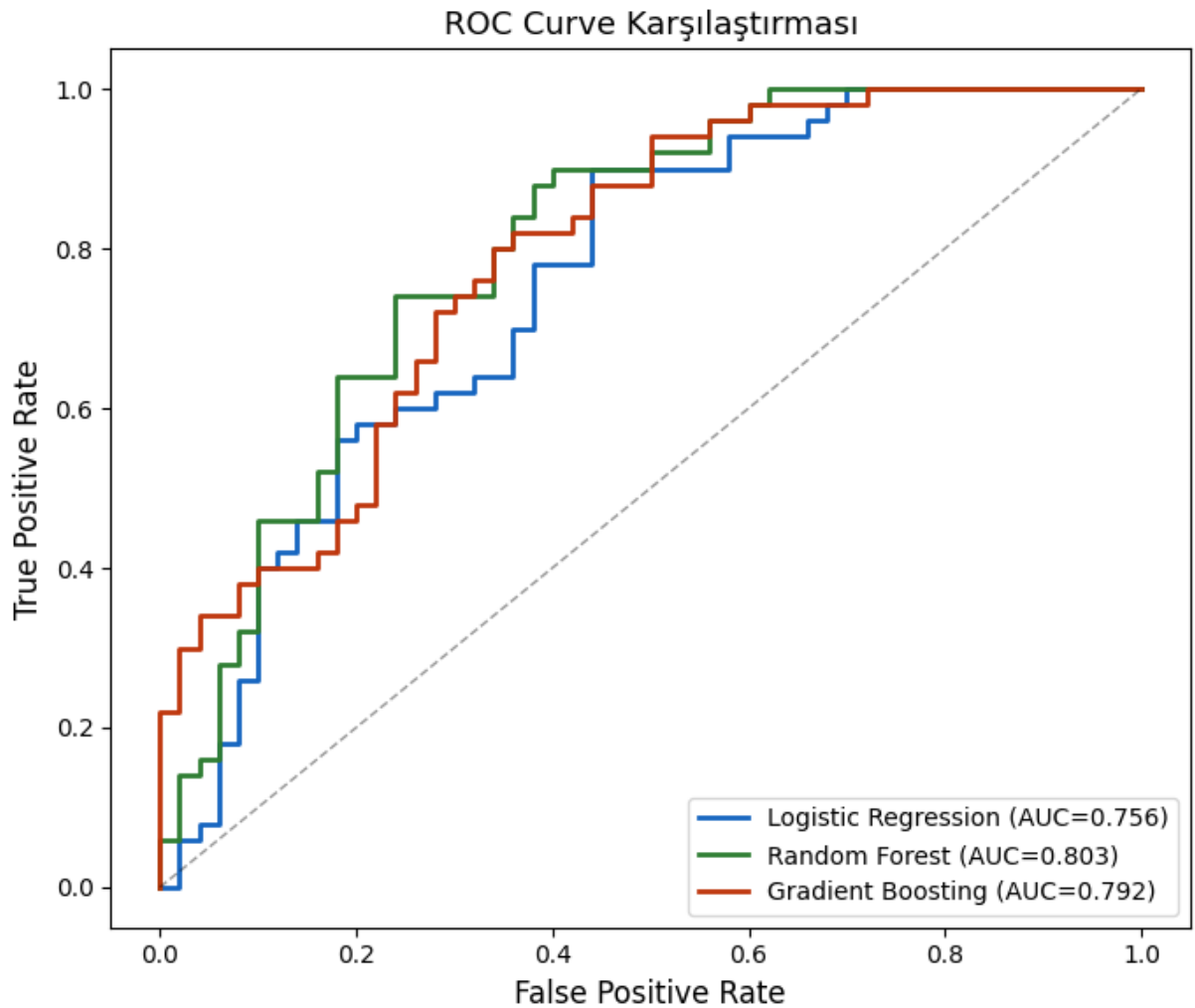
from sklearn.metrics import roc_curve

plt.figure(figsize=(7, 6))
colors = ['#1565C0', '#2E7D32', '#BF360C']

for (name, v), color in zip(ml_results.items(), colors):
    fpr, tpr, _ = roc_curve(y_test, v['y_prob'])
    plt.plot(
        fpr, tpr,
        lw=2,
        color=color,
        label=f"{name} (AUC={v['ROC-AUC']:.3f})"
    )

```

```
plt.plot([0, 1], [0, 1], 'k--', alpha=0.4, lw=1)
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC Curve Karşılaştırması', fontsize=13)
plt.legend(fontsize=10)
plt.tight_layout()
plt.savefig(f"{PATHS['figures']}/roc_curves_final.png", dpi=150)
plt.show()
```



```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

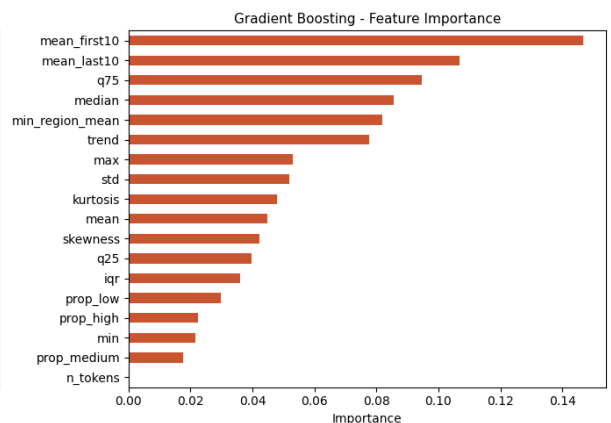
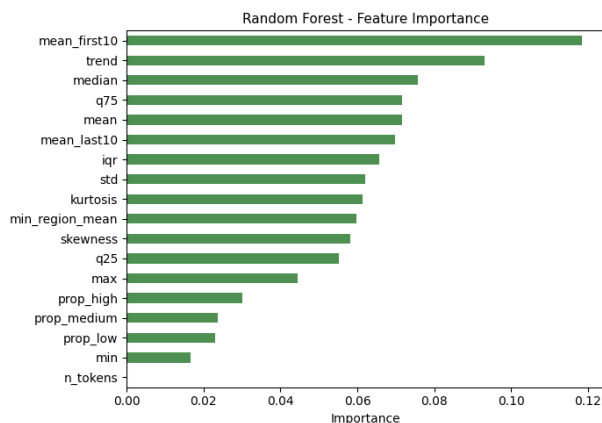
for ax, model_name, color in zip(
    axes,
    ['Random Forest', 'Gradient Boosting'],
    ['#2E7D32', '#BF360C']
):
    clf = ml_results[model_name]['clf'].named_steps['clf']
    imp = pd.Series(clf.feature_importances_, index=FEATURE_NAMES).sort_values(
        ascending=False
    )

    imp.plot(kind='barh', ax=ax, color=color, alpha=0.85)
    ax.set_title(f'{model_name} - Feature Importance', fontsize=11)
    ax.set_xlabel('Importance')

plt.tight_layout()
plt.savefig(f'{PATHS['figures']}/feature_importance_final.png", dpi=
plt.show())

print('\n=== En Önemli 5 Özellik (Gradient Boosting) ===')
gb_clf = ml_results['Gradient Boosting']['clf'].named_steps['clf']
gb_imp = pd.Series(gb_clf.feature_importances_, index=FEATURE_NAMES).sort_values(
    ascending=False
)
print(gb_imp.head(5).to_string())

```



```

=== En Önemli 5 Özellik (Gradient Boosting) ===
mean_first10      0.146739
mean_last10      0.106784
q75              0.094698
median           0.085590
min_region_mean  0.081803

```

```

from scipy import stats as sp_stats
import numpy as np

```

```

import pandas as pd

def extract_features(diffs):
    arr = np.array(diffs, dtype=float)
    n = len(arr)

    feat = {
        'mean'          : np.mean(arr),
        'median'        : np.median(arr),
        'std'           : np.std(arr),
        'min'           : np.min(arr),
        'max'           : np.max(arr),
        'q25'           : np.percentile(arr, 25),
        'q75'           : np.percentile(arr, 75),
        'iqr'           : np.percentile(arr, 75) - np.percentile(arr, 25),
        'skewness'       : float(sp_stats.skew(arr)),
        'kurtosis'       : float(sp_stats.kurtosis(arr)),
        'prop_high'      : np.mean(arr > 0.5),
        'prop_low'       : np.mean(arr < 0.1),
        'prop_medium'    : np.mean((arr >= 0.1) & (arr <= 0.5)),
        'mean_first10'   : np.mean(arr[:10]) if n >= 10 else np.mean(arr),
        'mean_last10'    : np.mean(arr[-10:]) if n >= 10 else np.mean(arr),
        'trend'          : (np.mean(arr[-10:]) - np.mean(arr[:10])) if n >= 20 else 0,
        'n_tokens'       : float(n),
        'min_region_mean': np.mean(arr[max(0, np.argmin(arr)-3):np.argmin(arr)+3])
    }

    return {
        k: float(np.nan_to_num(v, nan=0.0, posinf=0.0, neginf=0.0))
        for k, v in feat.items()
    }

def build_XY(df_subset):
    feats = [extract_features(row['top_diffs']) for _, row in df_subset.iterrows()]
    X = pd.DataFrame(feats)
    y = df_subset['is_correct'].astype(int).values
    return X, y

X_train, y_train = build_XY(train_df)
X_test, y_test = build_XY(test_df)

FEATURE_NAMES = X_train.columns.tolist()

print('Özellik sayısı:', len(FEATURE_NAMES))
print('X_train:', X_train.shape, 'X_test:', X_test.shape)

```

8-11-2026 10:10

```

uzellik sayisi: 18
X_train: (1000, 18)  X_test: (100, 18)

```

```

# =====
# FINAL ANALIZ - TEK HUCRE
# 1000 train + 100 test uzerinde ML + tablo + gorseller
# =====

import os, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from scipy import stats as sp_stats

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostin
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, ConfusionMatrixDisplay, roc_cur
)

matplotlib.rcParams["font.family"] = "DejaVu Sans"

# -----
# 1) Final checkpoint yükle
# -----
GEN_PATH_FINAL = f"{PATHS['generations']}/cosmos_t1_final_l4.json"

with open(GEN_PATH_FINAL, encoding="utf-8") as f:
    all_results = json.load(f)

df = pd.DataFrame(all_results)
df = df[df["top_diffs"].apply(lambda x: len(x) >= 5)].copy()

correct_df = df[df["is_correct"] == True].sample(frac=1, random_stat
wrong_df = df[df["is_correct"] == False].sample(frac=1, random_state

print(f"Kullanılabilir doğru: {len(correct_df)}")
print(f"Kullanılabilir yanlış: {len(wrong_df)}")

# -----
# 2) Final dengeli dataset
# -----
TRAIN_PER_CLASS = 500

```

```

TEST_PER_CLASS = 50
PER_CLASS = TRAIN_PER_CLASS + TEST_PER_CLASS

assert len(correct_df) >= PER_CLASS, f"Yeterli doğru yok! ({len(correct_df)} < {PER_CLASS})"
assert len(wrong_df) >= PER_CLASS, f"Yeterli yanlış yok! ({len(wrong_df)} < {PER_CLASS})"

train_df = pd.concat([
    correct_df.iloc[:TRAIN_PER_CLASS],
    wrong_df.iloc[:TRAIN_PER_CLASS],
]).sample(frac=1, random_state=42).reset_index(drop=True)

test_df = pd.concat([
    correct_df.iloc[TRAIN_PER_CLASS:PER_CLASS],
    wrong_df.iloc[TRAIN_PER_CLASS:PER_CLASS],
]).sample(frac=1, random_state=42).reset_index(drop=True)

print(f"\nEğitim seti: {len(train_df)} (doğru={train_df['is_correct'].sum()})")
print(f"Test seti: {len(test_df)} (doğru={test_df['is_correct'].sum()})")

train_df.to_json(f"{PATHS['data']}/train_balanced_final.json", orient='records')
test_df.to_json(f"{PATHS['data']}/test_balanced_final.json", orient='records')

# -----
# 3) Özellik çıkarma
# -----
def extract_features(diffs):
    arr = np.array(diffs, dtype=float)
    n = len(arr)

    feat = {
        "mean": np.mean(arr),
        "median": np.median(arr),
        "std": np.std(arr),
        "min": np.min(arr),
        "max": np.max(arr),
        "q25": np.percentile(arr, 25),
        "q75": np.percentile(arr, 75),
        "iqr": np.percentile(arr, 75) - np.percentile(arr, 25),
        "skewness": float(sp_stats.skew(arr)),
        "kurtosis": float(sp_stats.kurtosis(arr)),
        "prop_high": np.mean(arr > 0.5),
        "prop_low": np.mean(arr < 0.1),
        "prop_medium": np.mean((arr >= 0.1) & (arr <= 0.5)),
        "mean_first10": np.mean(arr[:10]) if n >= 10 else np.mean(arr),
        "mean_last10": np.mean(arr[-10:]) if n >= 10 else np.mean(arr),
        "trend": (np.mean(arr[-10:]) - np.mean(arr[:10])) if n >= 20 else 0,
        "n_tokens": float(n),
        "min_region_mean": np.mean(arr[max(0, np.argmin(arr)-3):np.argmax(arr)+3])
    }

```

```

    },

    return {
        k: float(np.nan_to_num(v, nan=0.0, posinf=0.0, neginf=0.0))
        for k, v in feat.items()
    }

def build_XY(df_subset):
    feats = [extract_features(row["top_diffs"]) for _, row in df_subset.iterrows()]
    X = pd.DataFrame(feats)
    y = df_subset["is_correct"].astype(int).values
    return X, y

X_train, y_train = build_XY(train_df)
X_test, y_test = build_XY(test_df)

FEATURE_NAMES = X_train.columns.tolist()

print("\nÖzellik sayısı:", len(FEATURE_NAMES))
print("X_train:", X_train.shape, " X_test:", X_test.shape)

# -----
# 4) ML modelleri
# -----
MODELS = {
    "Logistic Regression": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("sc", StandardScaler()),
        ("clf", LogisticRegression(max_iter=2000, C=1.0, random_state=42)),
    ]),
    "Random Forest": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("clf", RandomForestClassifier(
            n_estimators=300,
            max_depth=12,
            min_samples_leaf=3,
            random_state=42,
            n_jobs=-1,
        )),
    ]),
    "Gradient Boosting": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("clf", GradientBoostingClassifier(
            n_estimators=300,
            learning_rate=0.05,
            max_depth=5,
            subsample=0.8,
        ))
    ]),
}

```

```

        random_state=42,
    )),
    ]),
}

ml_results = {}

for name, clf in MODELS.items():
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    y_prob = clf.predict_proba(X_test)[:, 1]

    ml_results[name] = {
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall": recall_score(y_test, y_pred, zero_division=0),
        "F1": f1_score(y_test, y_pred, zero_division=0),
        "ROC-AUC": roc_auc_score(y_test, y_prob),
        "y_pred": y_pred,
        "y_prob": y_prob,
        "clf": clf,
    }

    print(
        f"{name}: Acc={ml_results[name]['Accuracy']:.4f}  "
        f"F1={ml_results[name]['F1']:.4f}  "
        f"AUC={ml_results[name]['ROC-AUC']:.4f}"
    )

# -----
# 5) Model karşılaştırma tablosu
# -----
tablo = pd.DataFrame({
    name: {
        "Accuracy": f"{v['Accuracy']:.4f}",
        "Precision": f"{v['Precision']:.4f}",
        "Recall": f"{v['Recall']:.4f}",
        "F1-score": f"{v['F1']:.4f}",
        "ROC-AUC": f"{v['ROC-AUC']:.4f}",
    }
    for name, v in ml_results.items()
}).T

print("\n" + "="*65)
print("          FINAL MODEL KARŞILAŞTIRMA TABLOSU")
print("="*65)
print(tablo.to_string())
print("="*65)

```

```

best = max(ml_results, key=lambda k: ml_results[k]["F1"])
print(f"\nEn iyi model (F1): {best} → F1={ml_results[best]['F1']}:.

# -----
# 6) Confusion Matrix
# -----
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, (name, v) in zip(axes, ml_results.items()):
    cm = confusion_matrix(y_test, v["y_pred"])
    ConfusionMatrixDisplay(cm, display_labels=["Yanlış", "Doğru"]).p
        ax=ax, colorbar=False, cmap="Blues"
    )
    ax.set_title(name, fontsize=11, fontweight="bold")

plt.suptitle("Final Confusion Matrix Karşılaştırması", fontsize=13,
plt.tight_layout()
plt.savefig(f"{PATHS['figures']}/confusion_matrices_final.png", dpi=
plt.show()

# -----
# 7) ROC Curve
# -----
plt.figure(figsize=(7, 6))
colors = ["#1565C0", "#2E7D32", "#BF360C"]

for (name, v), color in zip(ml_results.items(), colors):
    fpr, tpr, _ = roc_curve(y_test, v["y_prob"])
    plt.plot(
        fpr, tpr,
        lw=2,
        color=color,
        label=f"{name} (AUC={v['ROC-AUC']:.3f})"
    )

plt.plot([0, 1], [0, 1], "k--", alpha=0.4, lw=1)
plt.xlabel("False Positive Rate", fontsize=12)
plt.ylabel("True Positive Rate", fontsize=12)
plt.title("Final ROC Curve Karşılaştırması", fontsize=13)
plt.legend(fontsize=10)
plt.tight_layout()
plt.savefig(f"{PATHS['figures']}/roc_curves_final.png", dpi=150)
plt.show()

# -----
# 8) Feature Importance
# -----

```

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, model_name, color in zip(
    axes,
    ["Random Forest", "Gradient Boosting"],
    ["#2E7D32", "#BF360C"]
):
    clf = ml_results[model_name]["clf"].named_steps["clf"]
    imp = pd.Series(clf.feature_importances_, index=FEATURE_NAMES).sort_values(ascending=False)

    imp.plot(kind="barh", ax=ax, color=color, alpha=0.85)
    ax.set_title(f"{model_name} - Feature Importance", fontsize=11)
    ax.set_xlabel("Importance")

plt.tight_layout()
plt.savefig(f"{PATHS['figures']}/feature_importance_final.png", dpi=300)
plt.show()

print("\n=== En Önemli 5 Özellik (Gradient Boosting) ===")
gb_clf = ml_results["Gradient Boosting"]["clf"].named_steps["clf"]
gb_imp = pd.Series(gb_clf.feature_importances_, index=FEATURE_NAMES).sort_values(ascending=False)
print(gb_imp.head(5).to_string())

# -----
# 9) Summary JSON
# -----
summary = {
    "dataset": "ytu-ce-cosmos/gsm8k_tr",
    "model": "ytu-ce-cosmos/Turkish-Gemma-9b-T1",
    "n_train": len(train_df),
    "n_test": len(test_df),
    "features": FEATURE_NAMES,
    "results": {
        name: {
            k: round(v, 6)
            for k, v in vals.items()
            if k not in ("y_pred", "y_prob", "clf")
        }
        for name, vals in ml_results.items()
    },
    "best_model": best,
}

summary_path = f"{PATHS['results']}/summary_final.json"
with open(summary_path, "w", encoding="utf-8") as f:
    json.dump(summary, f, ensure_ascii=False, indent=2)

print("\nKaydedildi:", summary_path)

```

Kullanılabilir doğru: 551
Kullanılabilir yanlış: 1225

Eğitim seti: 1000 (doğru=500, yanlış=500)
Test seti: 100 (doğru=50, yanlış=50)

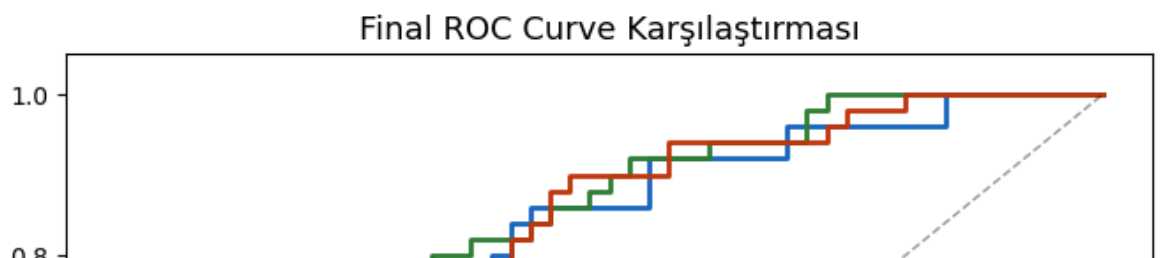
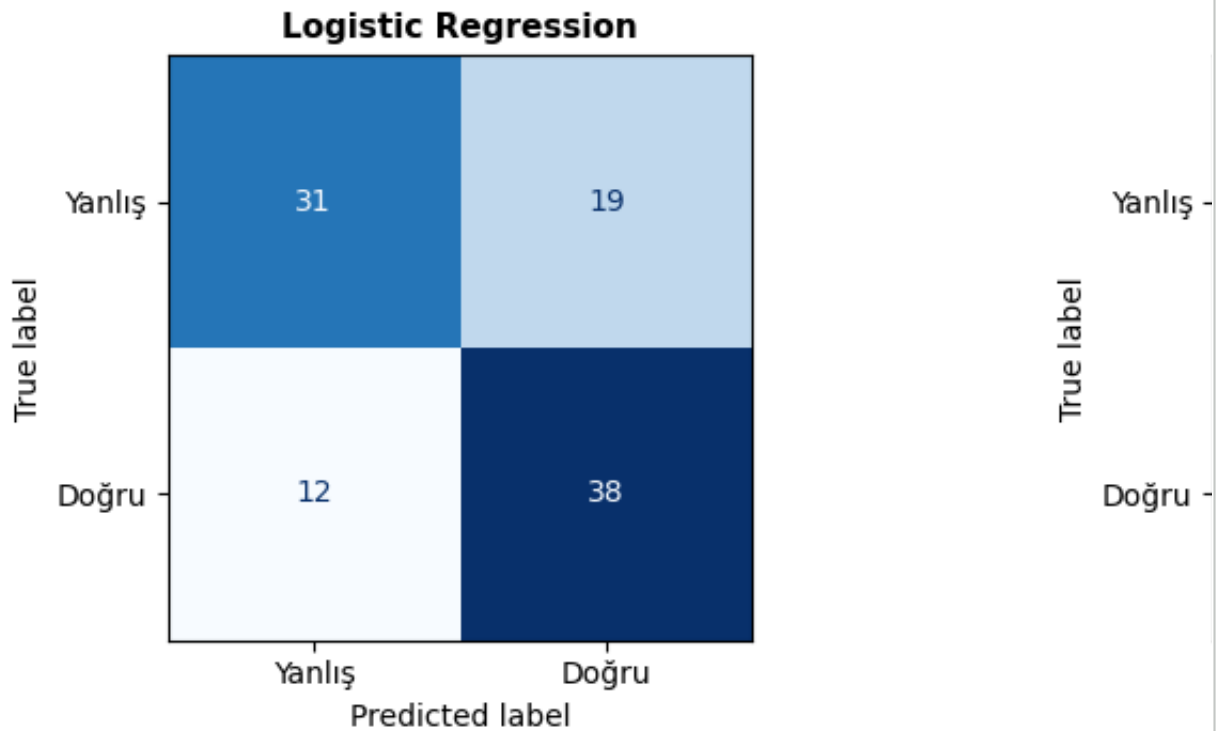
Özellik sayısı: 18
X_train: (1000, 18) X_test: (100, 18)
Logistic Regression: Acc=0.6900 F1=0.7103 AUC=0.7080
Random Forest: Acc=0.7300 F1=0.7429 AUC=0.7824
Gradient Boosting: Acc=0.6800 F1=0.6923 AUC=0.7936

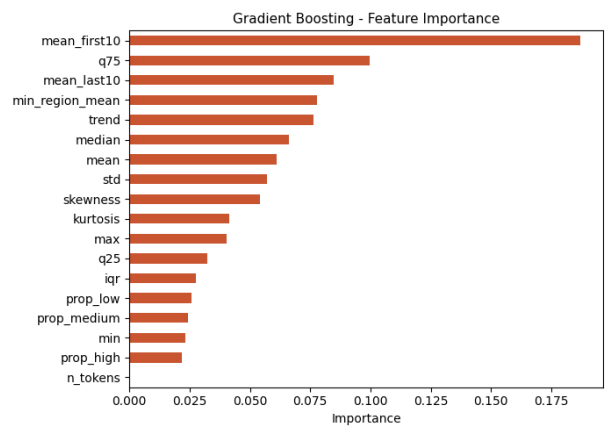
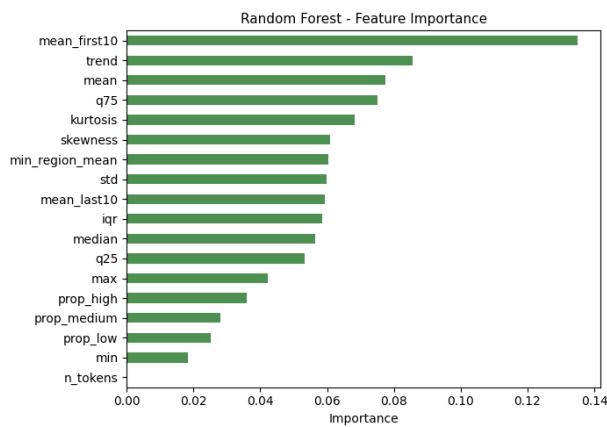
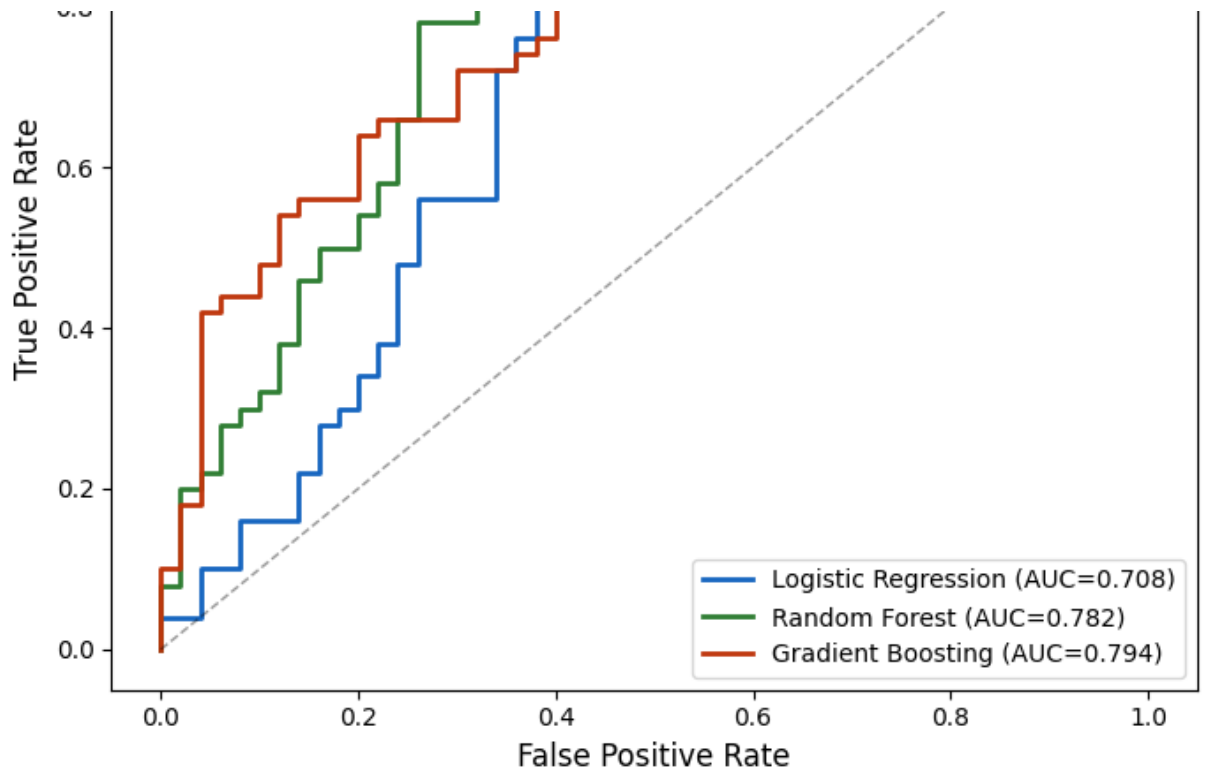
FINAL MODEL KARŞILAŞTIRMA TABLOSU

	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.6900	0.6667	0.7600	0.7103	0.7080
Random Forest	0.7300	0.7091	0.7800	0.7429	0.7824
Gradient Boosting	0.6800	0.6667	0.7200	0.6923	0.7936

En iyi model (F1): Random Forest → F1=0.7429

Final C





=== En Önemli 5 Özellik (Gradient Boosting) ===

mean_first10	0.187187
q75	0.099514
mean_last10	0.084568
min_region_mean	0.077824
trend	0.076305

Kaydedildi: /content/drive/MyDrive/top1top2_proje/results/summary_fi

